A Fast PC Algorithm for High Dimensional Causal Discovery with Multi-Core PCs

Thuc Duy Le, Tao Hoang, Jiuyong Li, Lin Liu, Huawen Liu, and Shu Hu

Abstract—Discovering causal relationships from observational data is a crucial problem and it has applications in many research areas. The PC algorithm is the state-of-the-art constraint based method for causal discovery. However, runtime of the PC algorithm, in the worst-case, is exponential to the number of nodes (variables), and thus it is inefficient when being applied to high dimensional data, e.g., gene expression datasets. On another note, the advancement of computer hardware in the last decade has resulted in the widespread availability of multi-core personal computers. There is a significant motivation for designing a parallelized PC algorithm that is suitable for personal computers and does not require end users' parallel computing knowledge beyond their competency in using the PC algorithm. In this paper, we develop parallel-PC, a fast and memory efficient PC algorithm using the parallel computing technique. We apply our method to a range of synthetic and real-world high dimensional datasets. Experimental results on a dataset from the DREAM 5 challenge show that the original PC algorithm could not produce any results after running more than 24 hours; meanwhile, our parallel-PC algorithm managed to finish within around 12 hours with a 4-core CPU computer, and less than six hours with a 8-core CPU computer. Furthermore, we integrate parallel-PC into a causal inference method for inferring miRNA-mRNA regulatory relationships. The experimental results show that parallel-PC helps improve both the efficiency and accuracy of the causal inference algorithm.

Index Terms—Causal discovery, gene expression data, high dimensional data, miRNA targets, parallel computing, PC algorithm

1 Introduction

Investigating the associations between variables has long been a main research topic in statistics, data mining and other research areas. For instance, to construct a gene regulatory network, we might want to find the associations between the expression levels of genes. We may use the observed association to explain that one gene regulates the expression of the other. However, the association does not tell us which gene is the regulator and which gene is the target, or indicate the chance of a third gene regulating the two genes. Hence, associations between genes may not imply the cause-effect nature of gene regulatory relationships.

There has been a long controversy over whether causality can be discovered from observational data. Many scientists have attempted to design algorithms of inferring causality from observational data over the last few decades [1], [2], [3], [4]. As a result, there has been more and more evidence demonstrating the possibility of discovering causal relationships from non-experimental data in different research areas [1], [5], [6]. Especially, two Nobel prizes were awarded for such causality methods in the field of Economics in 2003 and 2011 respectively [5], [6]. These works have strongly motivated the utilisation of observational data in exploring causal relationships in other important research areas, such as finding the genetic causes of cancers.

The standard method for discovering causality is a randomised controlled experiment. For example, to assess the effect of gene A on other genes, biologists use the gene knockdown experiment. In the experiment, they create two groups of samples, control and transfected groups. Gene A is knocked down in the control group whilst remains in the transfected group. The changes in the expression levels of the target genes between the two groups are the effects that gene A has on them. However, conducting such experiments would incur humongous costs given that thousands of genes are required to be tested. Moreover a randomised controlled experiment is generally impossible to conduct or restricted by ethical concerns in many cases. Therefore, discovering causality from observational data is a crucial problem.

One of the most advanced theories with widespread recognition in discovering causality is the Causal Bayesian Network (CBN). In this framework, causal relationships are represented with a Directed Acyclic Graph (DAG) [1]. Learning a DAG from data is highly challenging and complex as the number of possible DAGs is super-exponential to the number of nodes [7]. There are two main approaches for learning a DAG: the search and score approach, and the constraint based approach. While the search and score approach raises an NP-hard problem, the complexity of the constraint based approach is exponential to the number of nodes. The high computational complexity has hindered the applications of causal discovery approaches to high dimensional datasets, especially gene expression datasets of

- *T.D. Le, T. Hoang, J. Li, and L. Liu are with the School of Information Technology and Mathematical Sciences, University of South Australia, Mawson Lakes, SA 5095, Australia. E-mail: Thuc.Le@unisa.edu.au., hoatn002@mymail.unisa.edu.au, {jiuyong.li, lin.liu}@unisa.edu.au.*
- *H. Liu is with the Department of Computer Science, Zhejiang Normal University, Jinhua, Zhejiang 321004, China. E-mail: hwliu@zjnu.edu.cn.*
- *S. Hu is with the Department of Computer Science, USTC, Hefei, Anhui 230026, China. E-mail: huysy011@mymail.unisa.edu.au.*

Manuscript received 4 Nov. 2015; revised 23 May 2016; accepted 28 June 2016. Date of publication 14 July 2016; date of current version 7 Oct. 2019.
For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.
Digital Object Identifier no. 10.1109/TCBB.2016.2591526

1545-5963 © 2016 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: Univ Politecnica de Madrid. Downloaded on October 13,2026 at 21:42:58 UTC from IEEE Xplore. Restrictions apply.

which the number of genes (variables) is large and the number of samples is normally small.

A well-known constraint based algorithm is the Inductive Causation (IC) algorithm proposed in [8]. It is a conceptual algorithm and was implemented in the PC (named after its authors, Peter and Clark) algorithm [2]. The PC algorithm starts with a complete, undirected graph and deletes recursively edges based on conditional independence decisions. For example, the edge between A and B is removed if we can find a set S that does not include A and B, and when conditioning on S, A and B are independent. The PC algorithm has been implemented in various open-source sofware such as TETRAD [2], pcalg [9], and bnlearn [10], and has become a reliable tool for causal explorations.

There have been several applications of the PC algorithm in Bioinformatics [11], [12], [13], [14], [15]. Specifically, Zhang et al. [11] applied the PC algorithm which used mutual information for the conditional independence tests to learn the gene regulatory networks from gene expression data. Maathuis et al. [12] used the PC algorithm to learn the causal structure of the gene regulatory networks, then they applied *do-calculus* [1] to the learnt network to infer the causal effect that a gene has on the other. Le et al. [13] applied the approach in [12] to predict the targets of microRNAs (miRNAs, an important class of gene regulators at the post-transcriptional level). In a similar fashion, Zhang et al. [14] inferred the miRNA-mRNA interactions that had different causal effects between biological conditions, e.g., normal and cancer. These applications of the PC algorithm have been shown to outperform other computational methods that are based on correlation or regression analysis. Moreover, the PC algorithm is able to distinguish direct interactions from indirect interactions in gene regulatory networks [11], [15].

However, there are two main limitations of the PC algorithm, especially when applying to high dimensional biological datasets: (i) the runtime of the PC algorithm, in the worst case, is exponential to the number of nodes (variables), and thus it is inefficient when applying to high dimensional datasets such as gene expression datasets, and (ii) the result from the PC algorithm is variable ordered-dependent, i.e., when we change the order of the variables in the input dataset, the result may change.

Several methods [16], [17], [18], [19] have been proposed to improve the efficiency of the PC algorithm. They aim to introduce efficiency-improved alternative methods, but they rather compromise the accuracy. These methods either search for only specific causal structures or use heuristic functions to improve the efficiency of the algorithm. However, the results from these heuristic methods are either incomplete or involving a high rate of false discoveries.

Additionally, the order-dependence of the PC algorithm is problematic in high dimensional datasets. Colombo et al. [20] has shown experimentally that around 40 percent of the edges (2000 edges) learnt from a real gene expression dataset are not stable, i.e., these edges only appear in less than half of the results obtained with all the different orderings of nodes. This problem makes the knowledge inferred by using the PC algorithm less reliable. To overcome the order-dependence of the PC algorithm, researchers proposed several modifications to the procedure of the algorithm [2], [20], [21], [22]. For instance, Colombo et al. [20] proposed a modified version of the PC algorithm called stable-PC. The algorithm aims to query all the neighbours of each node and fix these neighbours while conducting conditional independence tests (CI tests henceforward) at each level (based on size of the conditioning sets) of the PC algorithm. However, this modification requires more CI tests and therefore results in even longer running time for this already inefficient algorithm. Our experiment on a dataset from the DREAM 5 challenge (http://dreamchallenges.org/) with 1643 variables and 805 samples shows that the order-independence version of the PC algorithm [20] takes more than 24 hours to run (unfinished). The inefficiency of the algorithm hinders its application in practice, as we would not estimate the runtime of a real world dataset up-front, and thus, prolonged running time would prevent us from applying the algorithm.

In another direction, parallel computing provides a straightforward approach to improving the efficiency of the algorithm, but it has been impractical for normal end users. The last decade has seen a fast growth of parallel computing frameworks, such as MapReduce framework [23]. However, such frameworks need suitable cluster facilities and require some solid technical understanding from users. Meanwhile, end users, e.g., biologists, normally use personal computers or small lab servers for their everyday research, and may not easily acquire the required technical knowledge for applying parallelised data mining algorithms. On the other hand, the fast development of computer hardware in the last decade has resulted in the widespread availability of multi-core personal computers. It is common for a modern computer to have a four-core CPU, and small servers to have eight-core CPUs. Therefore, it is of great interest to design parallel algorithms that are suitable for personal computers. Specifically, we aim to design the parallelised PC algorithm (called parallel-PC hereafter) for multi-core personal computers.

In this paper, we present a method to parallelise the order-independence PC algorithm in [20] for high dimensional datasets. We propose to parallelise the CI tests at each level of the algorithm. The CI tests at each level are grouped and distributed over different cores of the computer, and the results are integrated at the end of each level. Consequently, the runtime of our parallel-PC algorithm is much shorter than the original PC algorithm and the results of the parallel-PC algirhtm is independent of the variable ordering in a dataset. Importantly, it does not require any extra knowledge or installation effort from users. Our experiment results on both synthetic and real world datasets show that it is now practical to explore causal relationships with the PC algorithm in gene expression datasets using a multi-core personal computer. The proposed algorithm contributes to bridging the gap between computer science theory and its practicality in important scientific research areas. As an application, we modified the causal inference method in [13], by replacing the PC algorithm with the parallel-PC algorithm, to infer miRNA-mRNA regulatory relationships in three different cancer datasets. The experimental results show that our method not only outperforms the original causal inference method that uses the original PC algorithm, but also faster.

We summarise the contributions of the paper in the following:

1) Presenting a fast PC algorithm which is suitable for high dimensional datasets, and the results are

Authorized licensed use limited to: Univ Politecnica de Madrid. Downloaded on October 13,2026 at 21:42:58 UTC from IEEE Xplore. Restrictions apply.

independent of the order of the variables. To the best of our knowledge, this is the first work of utilising parallel computing for the PC algorithm.

2) Providing the software tool and proving that it is reliable and efficient using a wide range of real world gene expression datasets.
3) Modifying a causal inference method to infer miRNA-mRNA regulatory relationships with better performance.

The rest of the paper is organised as follows. Section 2 discusses related work, and Section 3 presents the PC algorithm and the stable-PC algorithm as well as the demonstrating examples. The proposed algorithm is presented in Section 4. Experiment results are shown in Section 5, Section 6 presents an application of parallel-PC in inferring miRNA-mRNA regulatory relationships, and finally Section 7 concludes the paper.

2 Related Work

The well-recognised methods for causality discovery are based on probabilistic graphical modeling [1], [24]. The structure of these graphical models is a DAG (directed acyclic graph), with its nodes representing random variables and edges representing dependencies between the variables [25]. There are two main approaches to learning the causal structure from data: 1) search and score and 2) constraint based methods. The search and score methods [24], [26], [27] search for all possible DAGs whilst using a scoring function to measure the fit of each DAG to the data. The DAG that best fits the data will be chosen. However, learning Bayesian networks using this approach is an NP-hard problem [28], and the proposed methods following this approach were only able to cater for datasets with a limited number of variables, and therefore may not be suitable for high dimensional datasets. Meanwhile, the constraint based approach [1], [16], [29] uses CI tests to remove non-causal relationships between variables. These methods are suitable for sparse datasets in practice [30].

Due to the significance of the PC algorithm in the constraint based causal discovery approach, there have been several methods aiming to modify the procedure of the PC algorithm directly to improve the efficiency and/or the accuracy of the algorithm. Steck and Tresp [31] used the necessary path condition to reduce the number of CI tests in the PC algorithm. They have proved that when testing the edge between X to Y, we do not need to condition on the nodes that are not in a path from X to Y. This modification may help reduce the chance for making errors by conducting fewer tests. However, it requires more running time for finding the nodes that are not in the path. Other researchers [29], [32] proposed to replace the CI test, which is normally based on the Chi-square statistical test, with the Bayesian statistical tests. They aim to reduce the error rates made by the CI tests. However, there is still no clear evidence in real world datasets about the impacts of this replacement on the efficiency improvement of the PC algorithm.

Meanwhile, the other constraint based methods [1] infer causal relationships by performing CI tests and searching for specific causal structures. For instance, Cooper [29] proposed the LCD method to learn only a small portion of edges rather than the complete network. Specifically, the LCD algorithm searches for the CCC structures which consist of three pairs of correlated variables: (A, B), (A, C) and (B, C). If A and C become independent when conditioned on B, we may infer that one of the following causal relations exists between A, B, and C: $A \leftarrow B \rightarrow C$; $A \rightarrow B \rightarrow C$; and $A \leftarrow B \leftarrow C$. Silverstein et al. [16] proposed the similar method that focuses on the CCU structures. The CCU method searches for two correlated pairs (A, B) and (A, C) and an independent pair (B, C). When conditioning on A, if (B, C) becomes dependent then the causal relationships $B \rightarrow A \leftarrow C$ are concluded. Compared to the PC algorithm, such specific structure finding methods are more efficient, but the causal discoveries are incomplete.

Due to the inefficiency of algorithms concerning causal relationships between all pairs of variables in a dataset, several methods have been proposed to discover only the local causal relationships from data, i.e., the relationships between a specific target and its neighbours. A popular approach is to identify the Markov blanket of a target variable. In a causal graph, Markov blanket has two equivalent definitions: 1) the Markov blanket of a node T is a set $MB(T)$ such that T is independent of other nodes when conditioning on $MB(T)$, or 2) the Markov blanket of a node T comprises its parents, children, and children's parents (spouses). Based on these two definitions, two main approaches have been proposed to discover the Markov blanket, the grow-and-shrink approach and the divide-and-conquer approach. The grow-and-shrink approach is based on the first definition and it uses the heuristic function in the "grow" step to find the superset of the true Markov blanket, then the false positives in this super set will be removed in the "shrink" step. Some well-known methods in this approach include GS (Grow and Shrink) [33], IAMB (Incremental Association Markov Blanket) [17], and IAMB variants [34]. The divide and conquer approach uses the topology constraints in the second definition of Markov blanket. This approach firstly finds the parents and children of the target node, then the spouses can be found by searching through the parents and children of the identified nodes in the first step. This approach attracts several methods namely HITON-PC/MB [35], MMPC-MB [36], semi-Interleaved HITON-PC [35], PCMB [37], and IPC-MB [38] (see [35] for a review).

In another direction, Jin et al. [39] proposed an efficient method for testing the persistent associations between the target variable and its neighbours. The method is based on partial association tests and association rule mining framework to remove the spurious associations. In a similar fashion, Li et al. [18], [19] integrated the ideas of retrospective cohort study with the association rule mining framework. They proposed to divide the samples into two groups of individuals, who share common characteristics but differ in regard to a certain factor of interest. The newly-designed dataset is called "fair dataset" and is used to infer the level of influence that the factor of interest affects the target variable. These works were extended in [40], and the software tools were also provided. However, these methods are only applicable for binary datasets with a fixed target variable, and thus may not be suitable for high dimensional datasets, e.g., gene expression data.

Authorized licensed use limited to: Univ Politecnica de Madrid. Downloaded on October 13,2026 at 21:42:58 UTC from IEEE Xplore. Restrictions apply.

The common characteristic of most of the above-mentioned works is that they aim to propose efficiency-improved alternative methods to the PC algorithm. However, the results from these heuristic methods may involve a high rate of false discoveries.

Scutari [41] utilised parallel computing technique for constraint based local causal discovery methods. Meanwhile, Chen et al. [42] proposed a method to parallelise the Bayesian network structure learning algorithm using search and score approach. However, there is still no existing work that parallelises the PC algorithm.

3 The Original-PC and Stable-PC Algorithms

3.1 Notation, Definitions, and Assumptions

Let $G = (\mathbf{V}, \mathbf{E})$ be a graph consisting of a set of vertices $\mathbf{V}$ and a set of edges $\mathbf{E} \subseteq \mathbf{V} \times \mathbf{V}$. The set of vertices that are adjacent to A in graph G is defined as: $adj(A, G) = \{B : (A, B) \in \mathbf{E}\ or (B, A) \in \mathbf{E}\}$. B is called a *collider* if we have the *v-structure*: $A \rightarrow B \leftarrow C$.

Graph G is a Directed Acyclic Graph (DAG) if G contains only directed edges and has no directed cycles. The *skeleton* of a DAG G is the undirected graph obtained from G by ignoring the direction of the edges. An *equivalence class* of DAGs is the set of DAGs which have the same skeleton and the same v-structures. An equivalence class of DAGs can be uniquely described by a *completed partially directed acyclic graph* (CPDAG) which includes both directed and undirected edges.

Definition 1. *(d-separation) [43]. In a DAG, a path p between vertices A and B is active (d-connecting, where "d" stands for dependence) relative to a set of vertices $\mathbf{C}$ ($A, B \notin \mathbf{C}$) if (i) every non-collider on p is not a member of $\mathbf{C}$; (ii) every collider on p is an ancestor of some member of $\mathbf{C}$. Two sets of variables $\mathbf{A}$ and $\mathbf{B}$ are said to be d-separated by $\mathbf{C}$ if there is no active path between any member of $\mathbf{A}$ and any member of $\mathbf{B}$ relative to $\mathbf{C}$.*

Let P be the joint probability distribution of $\mathbf{V}$. The following assumptions are set when applying the PC algorithm.

Assumption 1. *(Causal Markov Condition [2]) G and P satisfy the Causal Markov Condition if and only if given the set of all its parents, a node of G is probabilistically independent of all its non-descendents in G.*

Assumption 2. *(Faithfulness Condition [2]) G and P satisfy the Faithfulness Condition if and only if no conditional independence holds unless entailed by the Causal Markov Condition.*

Assumption 3. *(Causal sufficiency [2]) For every pair of variables which have their observed values in a given dataset, all their common causes also have observations in the dataset.*

3.2 The Original-PC Algorithm

The PC algorithm [2] (original-PC algorithm henceforward) has two main steps. In the first step, it learns from data a skeleton graph, which contains only undirected edges. In the second step, it orients the undirected edges to form an equivalence class of DAGs. As the first step of the PC algorithm contributes to most of the computational costs, we only focus on the modification of this skeleton learning step in this paper, and information about the edge orientation step can be found in [2].

The theoretical foundation of the original-PC algorithm [2] is that if there is no link (edge) between nodes X and Y, then there is a set of vertices $\mathbf{Z}$ that either are neighbours of X or Y such that X and Y are independent conditioning on $\mathbf{Z}$. In other words, $\mathbf{Z}$ disconnects X and Y. This foundation is presented as Theorem 1 below.

Theorem 1. *[2]. If vertices X and Y are not adjacent in a DAG G, then there is a set of vertices $\mathbf{Z}$ which is either a subset of $adj(X, G)\backslash Y$ or a subset of $adj(Y, G)\backslash X$ such that $\mathbf{Z}$ d-separates X and Y in G.*

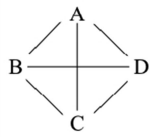
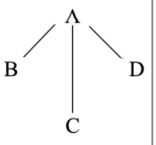
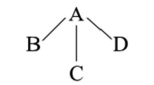
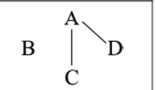
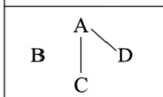
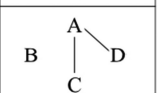
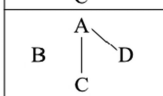
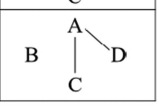
Based on Theorem 1, a naive approach for learning the skeleton is that we can search exhaustively for all possible neighbours of X and Y and perform conditional independence tests to justify if there is a set $\mathbf{Z}$ disconnecting X and Y. Obviously, this is an inefficient approach, as the neighbours of X and Y are unknown, and thus the search space is all variables other than X and Y. In the following we explain how the original-PC algorithm works and list the details of the skeleton learning step of the original-PC algorithm in Algorithm 1.

Algorithm 1. Step 1 of the Original-PC Algorithm: Learning the Skeleton

Input: Dataset D with a set of variables $\mathbf{V}$, and significant level α
Output: The undirected graph G with a set of edges $\mathbf{E}$
Assume all nodes are connected innitially
Let depth $d = 0$
repeat
  for *each ordered pair of adjacent vertices X and Y in G* **do**
    if ($|adj(X, G)\backslash\{Y\}| >= d$) **then**
      for *each subset $Z \subseteq adj(X, G)\backslash\{Y\}$ and $|Z| = d$* **do**
        Test $I(X, Y|Z)$
        if $I(X, Y|Z)$ **then**
          Remove edge between X and Y
          Save Z as the separating set of (X, Y)
          Update G and $\mathbf{E}$
          break
        end
      end
    end
  end
  Let $d = d + 1$
until $|adj(X, G)\backslash\{Y\}| < d$ *for every pair of adjacent vertices in G*;

In the skeleton learning step (Algorithm 1), the orginal-PC algorithm starts with the fully connected network and uses the conditional independence tests to decide if an edge is removed or retained. For each edge, the PC algorithm tests if the pair of variables connected by the edge, X and Y, are independent conditioning on a subset $\mathbf{Z}$ of all neighbours of X and Y. The CI tests are organised by levels (based on the size of the conditioning sets, e.g., the depth d). At the first level ($d = 0$), all pairs of vertices are tested conditioning on the empty set. Some of the edges would be deleted and the algorithm only tests the remaining edges in

Authorized licensed use limited to: Univ Politecnica de Madrid. Downloaded on October 13,2026 at 21:42:58 UTC from IEEE Xplore. Restrictions apply.

Level	Graph	#CI tests	Test	Result	Updated Graph
1		1	I(A, B)?	No	
		2	I(A, C)?	No	
		3	I(A, D)?	No	
		4	I(B, A)?	No	
		5	I(B, C)?	Yes	
		6	I(B, D)?	Yes	
		7	I(C, A)?	No	
		8	I(C, D)?	Yes	
		9	I(D, A)?	No	
2		10	I(A, B | C)?	Yes	
		11	I(A, C | D)?	No	
		12	I(A, D | C)?	No	

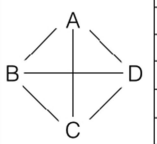
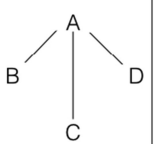
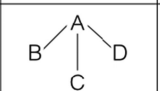
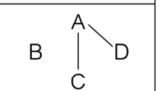
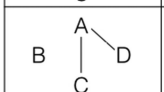
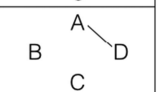
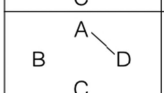
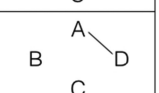
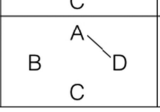
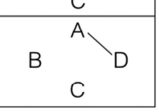
Level	Graph	# CI tests	Test	Result	Updated Graph
1		1	I(A, B)?	No	
		2	I(A, C)?	No	
		3	I(A, D)?	No	
		4	I(B, A)?	No	
		5	I(B, C)?	Yes	
		6	I(B, D)?	Yes	
		7	I(C, A)?	No	
		8	I(C, D)?	Yes	
		9	I(D, A)?	No	
2		10	I(A, B | C)?	Yes	
		11	I(A, C | B)?	Yes	
		12	I(A, D | B)?	No	
		13	I(A, D | C)?	No	

Fig. 1. Examples of applying PC (top) and stable-PC (bottom).

the next level ($d = 1$). The size of the conditioning set, d, is progressively increased (by one) at each new level until d is greater than the size of the adjacent sets of the testing vertices. The key feature that makes the PC algorithm efficient in sparse true underlying graphs is that the neighbours of a particular node are dynamically updated when an edge is removed. Therefore, the number of conditional independence tests is small when the true graph is sparse [2]. If the conditional independence tests are truly correct, then the PC algorithm will return the true graph as stated in Theorem 2 [2].

Theorem 2. *[2]. Let the distribution P be faithful to a DAG $G = (\mathbf{V}; \mathbf{E})$, and assume that we are given perfect conditional independence information about all pairs of variables (X, Y) in $\mathbf{V}$ given subsets S. Then the output of the PC-algorithm is the CPDAG that represents G.*

Fig. 1 (top) shows an example of original-PC algorithm being applied to a dataset with four nodes, A, B, C, and D. The original-PC algorithm starts with the fully connected graph. At the first level, all edges are tested conditioning on the empty set. In the example, after level 1 tests, there are three edges left. At the next level, each remaining edge will be tested conditioning on each neighbour (size 1) of the testing variables. For example, with the edge between A and B we have at most two tests which are conditioning on C and conditioning on D. If the test returns independence (e.g., $I(A, B|C)$), we remove the edge from the graph and move to testing the other edge. The procedure will stop when there is no test to perform.

3.3 The Stable-PC Algorithm

The CI tests in the original-PC algorithm are prone to mistakes as the number of samples is limited. Moreover, incorrectly removing or retaining an edge would result in the changes in the neighbour sets of other nodes, as the graph is updated dynamically. Therefore, the output graph is dependent on the order in which we perform the conditional independence tests. In other words, given a lexical order of performing the tests, the order of the variables in the input data will affect the output graph. In Fig. 1 (top), at the beginning of level 2, after we perform test $\#10$, the edge between A and B is removed and the graph is updated. This also means that the adjacent set of A is also updated, i.e., $adj(A, G) = \{C, D\}$. Therefore, when we test the edge between A and C, we do not need to condition on B. The updates make the original-PC algorithm efficient, but they also cause problems when the CI tests are making mistakes. If test $\#10$ wrongly removes the edge between A and B, then we miss the test $I(A, C|B)$ which may remove the edge between A and C. Moreover, if we test $I(A, C|B)$ first and remove the edge between A and C, we will end up with a different graph. In other words, when we change the order of the variables in the dataset, e.g., A, C, B, and D, the result of the original-PC algorithm may change.

Colombo et al. [20] proposed a modification to the original-PC algorithm to obtain a stable output skeleton which does not depend on how variables are ordered in the input dataset. In this method (called stable-PC algorithm), the neighbour (adjacent) sets of all nodes are queried and kept unchanged at each particular level. As a result, an edge deletion at one level does not affect the conditioning sets of the other nodes, and thus the output is independent with the variable ordering. For instance, at the beginning of level 2 in the above example, the adjacent set of node A is $adj(A, G) = \{B, C, D\}$. This adjacent set is kept unchanged at this level. After performing test $\#10$, we remove the edge between A and B in the graph, but the $adj(A, G)$ is not updated. Therefore, we still consider B as an adjacent node of A, and tests $\#11$ and $\#12$, still need to be performed as shown in Fig. 1 (bottom). Test $\#11$ removes the edge between A and C, and therefore generates different output graph compared to the original-PC algorithm.

However, this modification requires performing more conditional independence tests in each level of the algorithm, and thus it increases further the runtime of the algorithm. In the following section, we adopt this modification to design a modified PC algorithm which is both efficient and order-independent.

Authorized licensed use limited to: Univ Politecnica de Madrid. Downloaded on October 13,2026 at 21:42:58 UTC from IEEE Xplore. Restrictions apply.

4 Parallel-PC Algorithm

4.1 Parallel Computing: Potentials and Challenges

Computer hardware technology has been advanced significantly in the last decade. Nowadays, it is common to have personal desktop computers and even laptops have four cores in their CPUs. Likewise, the number of cores in today's servers has jumped to 8, 16, 32 or more. These resources provide a great potential of utilising parallel computing for important algorithms, including the PC algorithm, for personal computers. In fact, the majority of end users are using their personal computers for doing research. Therefore, parallel algorithms would have significant impact if they work well for personal computers and do not require users' technical knowledge about parallelism and high-performance clusters.

The idea of parallelism is that we break down a big task into several different smaller subtasks and distribute them over different cores of the computer's CPU to run in parallel. The results from all subtasks will then be integrated to form the result of the original task. Theoretically, this divide-and-conquer approach can speed up an algorithm by the maximum number-of-core times. This approach, however, requires the subtasks to be unrelated. In other words, the result from each subtask must be independent of one another and no communication is allowed between them.

The independence requirement has been a great barrier for many sequential algorithms, including the stable-PC algorithm, to be parallelised. In the stable-PC algorithm, the performance bottleneck is the huge number of CI tests, which can reach hundreds of millions in real-world datasets. The CI tests, however, are order-dependent across different levels. Since the stable-PC algorithm updates the adjacent sets of all nodes at each level after all the CI tests of the level are completed, the test results of a particular level will influence the results of the next level. Therefore, it seems infeasible to parallelise the stable-PC algorithm as a whole.

4.2 The Parallel-PC Algorithm

Our proposed strategy is to parallelise the CI tests, not across different levels, but inside each level of the stable PC-algorithm. This approach is feasible because the CI tests at a particular level are independent of each other. Since the graph is only updated at the end of each level, the result of one CI test does not affect the others. Therefore, the CI tests at a level can be executed in parallel without changing the final result. Furthermore, this approach also enjoys the advantage of knowing the number of CI tests of each level in advance. This allows the CI tests to be evenly distributed over different cores, so that the parallelised algorithm can achieve maximum possible speedup.

Within a level (depth) of Algorithm 2, we employ the parallel computing paradigm to parallelise their executions (the parallelisation step). At each level with the conditioning sets of size d, the sequential process in the stable-PC algorithm is replaced by a three-staged process: (1) the CI tests are distributed evenly among the cores, (2) each core performs its own sets of CI tests in parallel with the others, and (3) the test results in all cores are integrated into the global graph. This three-staged process is applied at all the levels (the depth d) of the algorithm.

Algorithm 2. The Parallel-PC Algorithm

Input: Dataset D, significant level α, P cores, memory-efficient indicator s, number of edges per batch t_b
Output: The undirected graph G with a set of edges E
Assume all nodes are connected in graph G
Let depth $d = 0$
repeat
  Query and fix the adjacent set $adj(X, G)$ of each node X in G
  Compute the set J of unordered pairs of adjacent vertices (X, Y) in G
  // `Parallelisation Step`
  for *each batch of* t_b *edges* ($t_b = |J|$ *if* $s = FALSE$) **do**
    Distribute the edges in the batch evenly into P cores, each with J_p edges
    for *each core* $p = 1P$ *in parallel* **do**
      for *each pair* $(X, Y) \in J_p$ **do**
        Let $k^p_{X,Y}$ indicate if (X, Y) is adjacent,
        initialize $k^p_{X,Y} = \text{TRUE}$
        // On X's `neighbours`
        if $|adj(X, G)\backslash\{Y\}| \geq d$ **then**
          for *each subset* $Z_X \subseteq adj(X, G)\backslash\{Y\}$ *and* $|Z_X| = d$ **do**
            if $I(X, Y|Z_X)$ **then**
              $k^p_{X,Y} = \text{FALSE}$
              break
            end
          end
        end
        // On Y's `neighbours`
        if $|adj(Y, G)\backslash\{X\}| \geq d$ **then**
          for *each subset* $Z_Y \subseteq adj(Y, G)\backslash\{X\}$ *and* $|Z_Y| = d$ **do**
            if $I(X, Y|Z_Y)$ **then**
              $k^p_{X,Y} = \text{FALSE}$
              break
            end
          end
        end
      end
    end
    // `Synchronisation Step`
    for *each core* $p = 1 \ldots P$ **do**
      for *each pair* $(X, Y) \in J_p$ **do**
        if $k^p_{X,Y} = \text{FALSE}$ **then**
          Remove the edge between X and Y and update G and E
        end
      end
    end
  end
  Let $d = d + 1$
until $|adj(X, G)\backslash\{Y\}| < d$ *for every pair of adjacent vertices in* G;

We observed that it is not efficient to distribute the CI tests of the same edge to different cores. Given an edge between X and Y, we need to perform the CI tests conditioning on the variables in $adj(X, G)$ and in $adj(Y, G)$. However, if we firstly perform the CI tests conditioning on variables in $adj(X, G)$ and the edge between X and Y is removed, then the CI tests conditioning on variables in

Authorized licensed use limited to: Univ Politecnica de Madrid. Downloaded on October 13,2026 at 21:42:58 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
Real World Gene Expression Datasets

Dataset	#samples	#variables	stable-PC runtime
NCI-60	47	1,190	13.56 mins
MCC	88	1,380	59.87 mins
BR51	50	1,592	68.37 mins
S. cerevisiae	63	5,361	207.41 mins
S. aureus	160	2,810	477.20 mins
DREAM5-Insilico	805	1,643	$>$ 24 hours

$adj(Y, G)$ are unnecessary. We resolve this dependency by grouping the CI tests of the same edge together, instead of separating them as in the stable-PC algorithm. In Algorithm 2, if the CI tests between X and Y return independence when conditioning on $\mathbf{Z} \subset adj(X, G)\backslash\{Y\}$, the algorithm will not perform the CI tests conditioning on variables in $adj(Y, G)$. Grouping those tests together, the algorithm ensures the CI tests of the same edge will not be distributed to different cores of the CPU, and thus reduce the number of unnecessary CI tests that need to be performed.

In Algorithm 2, we provide an optional input parameter, memory-efficient indicator s. If s is set to TRUE, the algorithm will detect the free memory of the running computer to estimate the number of edges in G that will be distributed evenly to the cores. This step is to ensure that each core of the computer will not hold off a big amount of memory while waiting for the synchronisation step. The memory-efficient procedure may consume a little bit more time. However, this option is recommended for computers with limited memory resources or for big datasets.

4.3 Implementation

We implement our parallel-PC algorithm using R [44]. Our implementation is based on the stable-PC algorithm in the R package *pcalg* [9] and the native R library *parallel*. The implementation has been tailored to work in Linux, MacOS, and Windows operating systems. The codes of parallel-PC with and without the memory efficient option together with the instructions of running the algorithm are available at: http://nugget.unisa.edu.au/ParallelPC

5 Experimental Evaluation and Applications

5.1 Datasets

In this section, we evaluate the efficiency of the proposed algorithm by applying it to five real world datasets and a synthetic dataset (see Table 1 for a summary of the datasets). The first three datasets are commonly used in research of inferring the regulatory relationships between microRNAs (a type of gene regulators) and genes. These datasets were downloaded from [45]. Meanwhile, the last three datasets are used for discovering the regulatory relationships between transcription factors (another type of gene regulators) and the target genes regulated by them. The *S. cerevisiae* dataset was provided by the authors of [12], and the last two datasets were downloaded from [46]. The number of samples and variables in each dataset together with the runtime of the stable-PC algorithm are summarised in Table 1. We choose these datasets for the experiments, as they require a wide range of runtime when applying the stable-PC algorithm. All of the datasets are available at: nugget.unisa.edu.au/ParallelPC

5.2 Efficiency Evaluations

We apply the stable-PC algorithm, and the parallel-PC algorithm to all of the datasets and compare their performance in terms of the runtime. For each dataset, we run five versions of the parallel-PC algorithm using 4, 5, 6, 7, and 8 cores CPU respectively. Meanwhile, for the stable-PC algorithm, we used the implementation in the *pcalg*[9] R package. We choose the range from four to eight cores for the experiments, as a modern personal computer normally has four cores and a small server usually has eight cores. We run all the experiments on a Linux server with 2.7 GB memory and 2.6 GHz per core CPU. All experiments were run three times and the average value of the three runs was reported.

As shown in Fig. 2, the runtime of the parallel-PC algorithm is much shorter compared to the stable-PC algorithm. Please note that the parallel-PC algorithm generates the results that are independent of the order of the variables in the datasets, and the output results are consistent with the stable-PC algorithm.

In the relatively small datasets, NCI-60, stable-PC took around 14 minutes to complete, but parallel-PC with 4-core CPU only spent around 5 minutes. For this small dataset, increasing the number of cores does not reduce the runtime significantly due to the cost of synchronisation between the cores.

For the BR51 and MCC datasets, it took more than one hour each for running the stable-PC algorithm. The 6-core parallel-PC algorithm only used less than one third (less than 20 minutes) of its counterpart's runtime. Meanwhile, there is a clear trend in the *S. cerevisiae* dataset that the more powerful CPU (up to 15 cores) we use, the shorter runtime the parallel-PC algorithm has.

The efficiency of the parallel-PC algorithm is even higher with big datasets. In the S.aureus dataset, the runtime of the stable-PC algorithm was around 8 hours. Meanwhile, using a 4-core version of the parallel-PC algorithm, it took only around two hours. This is a significant result for end users who only use personal computers for doing their research experiments. The runtime is further reduced when using computer CPU with more cores. For instance, it took just more than one hour (76.1 minutes) for running this dataset with an 8-core CPU computer (server). With the DREAM5-Insilico dataset, after more than 24 hours the stable-PC still could not produce any results, and we had to abort the job. This is a typical situation when applying the stable-PC algorithm to big datasets, as we may not be able to predict its running time on a real world dataset. However, our 4-core parallel-PC algorithm managed to finish within around 12 hours, and the 8-core version took less than 6 hours (341.2 minutes).

The experiment results suggest that our proposed algorithm has significantly shorter runtime than the stable-PC algorithm. Given that our method generates the same output and requires no extra effort from users compared to that of the stable-PC algorithm, the parallel-PC is more useful in exploring causal relations in high dimensional data, especially gene expression data. Note that the provided

Authorized licensed use limited to: Univ Politecnica de Madrid. Downloaded on October 13,2026 at 21:42:58 UTC from IEEE Xplore. Restrictions apply.

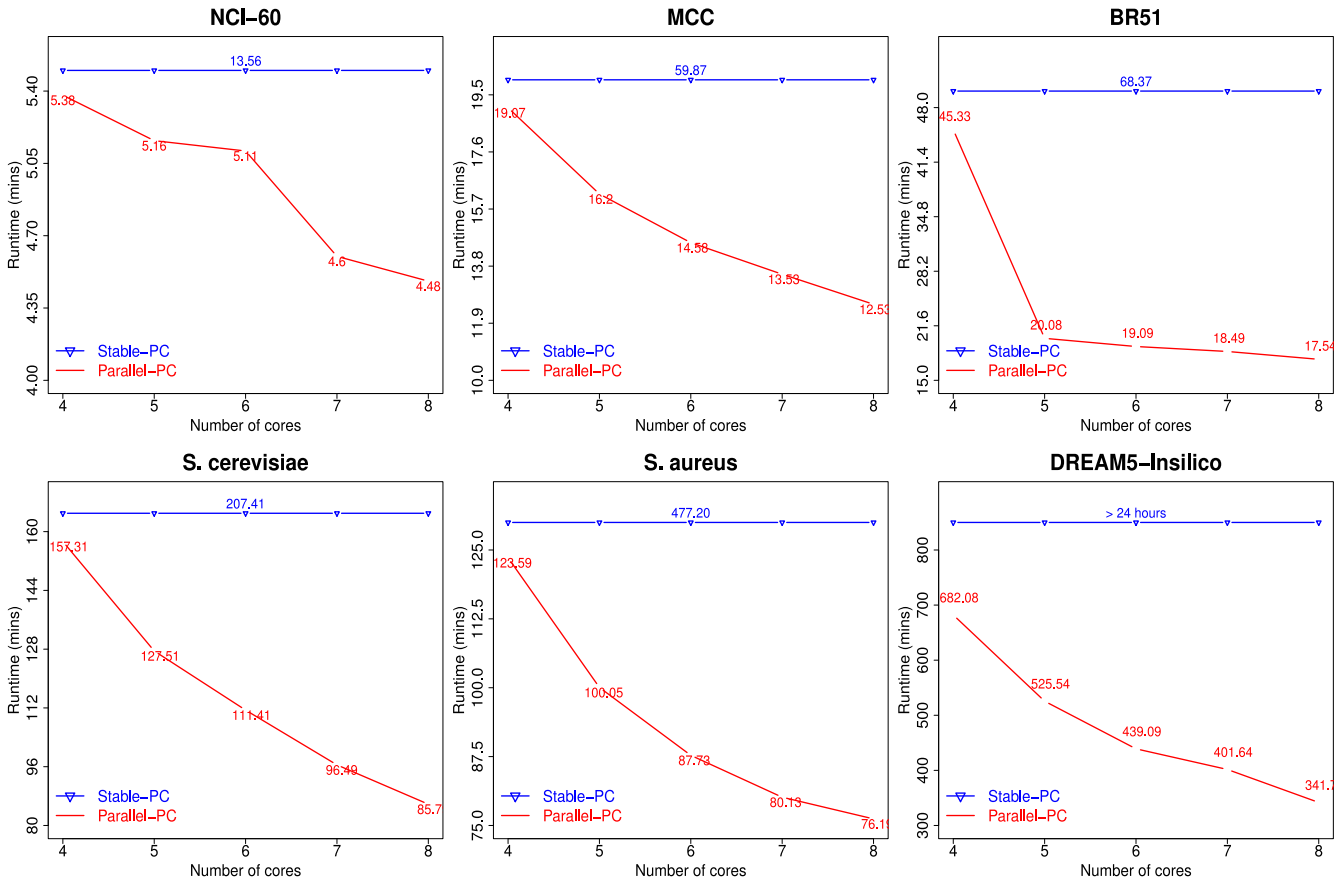


Fig. 2. Runtime of the algorithms on the six datasets. We run each experiment three times, and each value in the figure is the average runtime of the three runs. The average variation coefficient (standard deviation / mean) of the PC-parallel algorithm in the three runs is 0.039 and the maximum variation coefficient is 0.055.

software tool is not restricted to only personal PCs, and it is ready for clusters or super computers with CPUs of more cores. To provide more insight into the experiments in Fig. 2, we run the experiments on a super computer (with a 48-core CPU) using up to 16 cores of the CPU and the results are in Fig. 3. For most of the datasets, the runtime was not improved further when we used more than 14 cores of the CPU.

5.3 Scalability

To compare the scalability of the parallel-PC algorithm and the stable-PC algorithm, we used the DREAM5-insilico dataset as a base and randomly sample from it four datasets with different numbers of samples (200, 400, 600, and 805) and five datasets with different numbers of variables (500, 750, 1,000, 1,250, and 1,500).

Fig. 4 (top) shows that the runtime of the stable-PC algorithm increases significantly when the number of variables is increased in a dataset. The reason is that the number of CI tests is increased when the number of variables is increased. The parallel-PC algorithm suffers from the same problem, but the increasing trend of the runtime is less significant compared to that of the stable-PC algorithm.

Similarly, the runtime of both PC and parallel-PC goes up when we increase the number of samples in the dataset as shown in Fig. 4 (bottom). However, the reason for runtime increase is not because of the increased number of CI tests, but due to the increase in the time taken to perform a test. When we increase the number of samples in the input dataset, the time for performing one CI test is increased, and thus resulting in the large increase in total runtime given that the number of CI tests is huge.

5.4 Impact of the Memory-Efficient Option

As mentioned in Section 4, the memory-efficient version of the algorithm uses less amount of memory at any time point while running the algorithm. We observed that peak memory usage in the algorithm with memory-efficient option (s=TRUE) is about half of that of the algorithm without using the option across the six datasets in Table 1. However, there is a trade-off between memory efficiency and time efficiency as show in Table 2. The experiments were conducted on a MacBook Pro with 8 GB RAM, 2.6 GHz. Running *S. cerevisiae* with $s = FALSE$ failed to finish due to memory exhaustion.

6 Application in Inferring miRNA-mRNA Regulatory Relationships

6.1 Computational Methods for Identifying miRNA Targets

microRNAs (miRNAs) are important gene regulators at post-transcriptional level. They regulate gene expression by promoting messenger RNA (mRNA) degradation and repressing translation [47]. miRNAs control a wide range of biological processes and are involved in several types of

Authorized licensed use limited to: Univ Politecnica de Madrid. Downloaded on October 13,2026 at 21:42:58 UTC from IEEE Xplore. Restrictions apply.

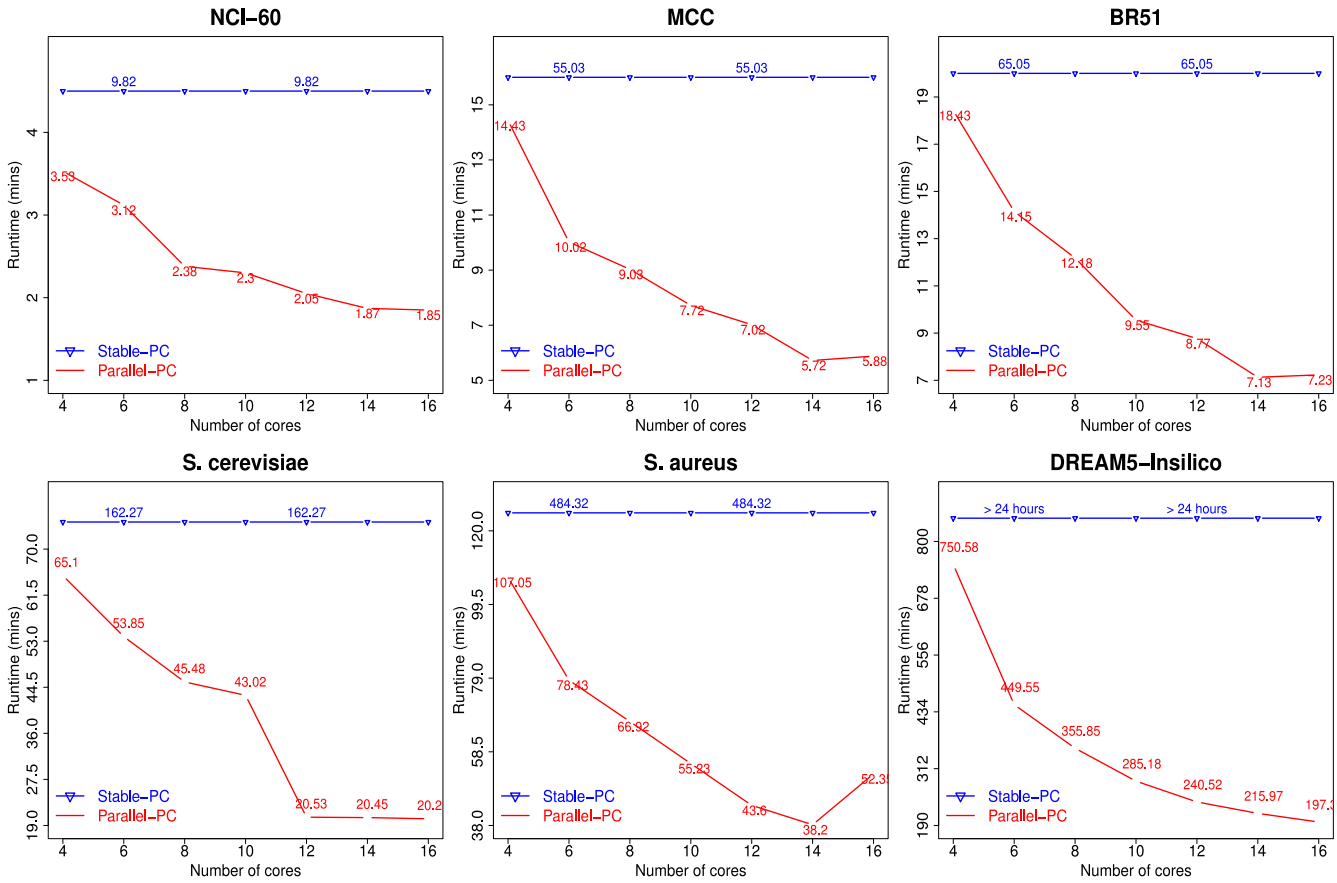


Fig. 3. Runtime of the algorithms using up to 16 cores with the six datasets.

cancers [47], [48]. In the last decade, predictions of miRNA functions through identifying miRNA-mRNA regulatory relationships by computational methods have increasingly achieved promising results. Computational approaches are proving to be effective in generating hypotheses to assist with the design of wet-lab experiments for confirming miRNA targets.

Several methods have been proposed to identify miRNA-mRNA regulatory relationships. In the first stream, researchers identified miRNA-mRNA interactions using sequence data [49], [50], [51]. Although these methods can predict potential miRNA targets, the results may contain a high-rate of false positives and false negatives [52]. In the second stream of research, various computational methods have been devised to use expression profiles in the study of miRNA-mRNA regulatory relationships. The principle of these methods is to investigate if a change in the miRNA expression level would result in a change in the mRNA expression. Some highlights of the methods are correlation analysis [53], [54], regression models [55], [56], population-based probabilistic learning model [57], rule based methods [58], Bayesian network learning [59], [60], [61], causal inference techniques [13], [14], [15], and ensemble methods [62]. Please refer to [45] for a survey of different miRNA target prediction methods.

6.2 Inferring miRNA-mRNA Regulatory Relationships with efficient-IDA

In this section, we propose to use parallel-PC to modify the the causal inference method [13] for inferring miRNA-mRNA regulatory relationships. The method is based on a causal inference approach called IDA (Intervention effect when the DAG is absent) [63]. IDA firstly learns the causal structure from data using the PC algorithm, it then uses *do-calculus* to estimate the causal effect that a variable has on the other. The estimated causal effects simulate the effects of randomised controlled experiments. Although the method is proved to be effective in uncovering miRNA-mRNA regulatory relationships, it has high computational complexity and suffers from the order-dependent problem. In this section, we propose to tackle these problems by modifying the IDA-based method by using parallel-PC rather than PC as the first step of the IDA algorithm. The modified algorithm is called efficient-IDA and includes the following steps.

- Step 1: Find miRNAs and mRNAs that are differentially expressed between different conditions, e.g., normal vs cancer. The expression profiles of those miRNAs and mRNAs will be combined into a dataset where rows are samples and columns are variables (miRNAs and mRNAs). In our experiments, we download from [45] three processed cancer datasets with differentially expressed profiles.
- Step 2: Apply parallel-PC to learn the causal structure from data. parallel-PC will help tackle the efficiency and order-dependency problems of the original PC algorithm.
- Step 3: Use *do-calculus* to estimate the causal effects that the miRNAs have on the mRNAs. For more information on this step, please refer to [13].

Authorized licensed use limited to: Univ Politecnica de Madrid. Downloaded on October 13,2026 at 21:42:58 UTC from IEEE Xplore. Restrictions apply.

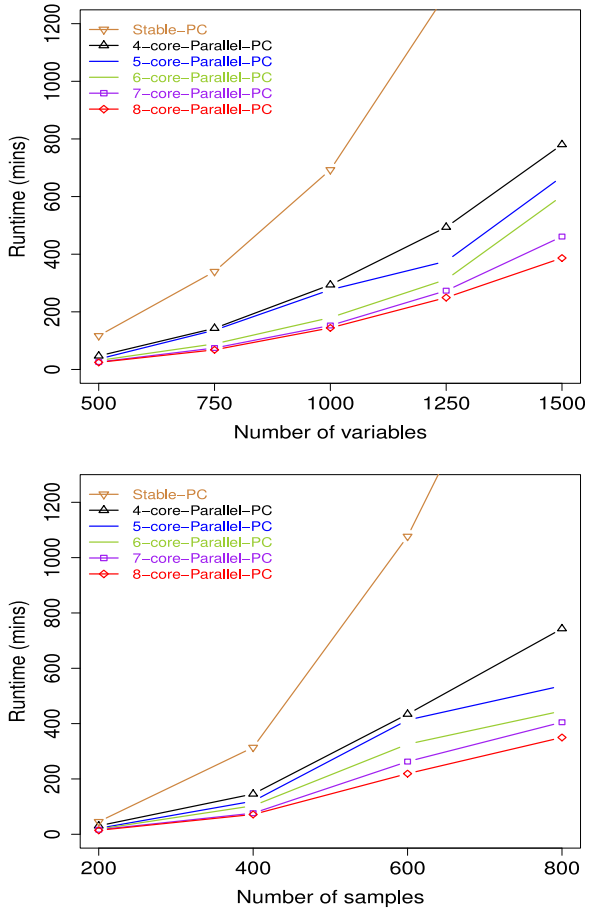


Fig. 4. Scalibility of the parallel-PC algorithm with number of attributes (top) and with data size (bottom).

The running time of IDA is quite similar to the PC algorithm, as the time for the inference step (Step 3 as described above) is small. Therefore we do not compare the runtime of efficient-IDA and IDA again and please refer to Figs. 2 and 3 for details. In the following sub-sections, we compare the performance of efficient-IDA and IDA in correctly identifying miRNA-mRNA regualtory relationships.

6.3 efficient-IDA Discovers More Experimentally miRNA Targets than IDA

We applied efficient-IDA and IDA to NCI-60, MCC, and BR51 and compare their performance on discovering the experimentally confirmed miRNA targets.

To validate the target predictions from the two methods, we used the combination of four different experimentally confirmed miRNA target databases as the ground truth. They are Tarbase v6.0 [64], miRecords v2013 [65], miRWalk v2.0 [66], and miRTarBase v4.5 [67]. Tarbase, miRecords, and miRTarBase include verified interactions that are manually curated from the literature, and miRWalk contains both the predicted and the experimentally validated interactions. Respectively for Tarbase, miRecords, miRWalk, and miRTarBase, we have 20,095 interactions with 228 miRNAs, 21,590 interactions with 195 miRNAs, 1,710 interactions with 226 miRNAs, and 37,372 interactions with 576 miRNAs. After removing the duplicates, we have in total 62,858 unique interactions to be used in the validation.

TABLE 2
Impact of the Memory-Efficient Option

Dataset	Time (mins)		Memory (MBs)	
	$s = F$	$s = T$	$s = F$	$s = T$
NCI-60	2.2	2.9	919	421.5
MCC	9.4	11.6	1,323	533.5
BR51	11.9	15.3	1,593	679.4
Insilico	531.7	557.9	1,533	784
S. aureus	92.1	91.3	4,038	1,857.1
S. cerevisiae	NA	91.3	$>$8,000	6,004

We used two different ways to extract prediction results for the comparison. First, we extracted the top 100 predicted targets of each of the miRNAs in a dataset and validate them against the experimentally confirmed miRNA targets. Secondly, we extracted the top 1,000, 2,000, 5,000, and 10,000 predicted miRNA-mRNA interactions of each method for validation.

Fig. 5 shows the comparison results of the number of confirmed miRNA targets of efficient-IDA and IDA using the two validation methods respectively. We can see from the figure that efficient-IDA discovers more confirmed miRNA targets than IDA in both of the cases in all datasets. The experiment results suggest that efficient-IDA is not just faster in runtime, but the order-independent property of

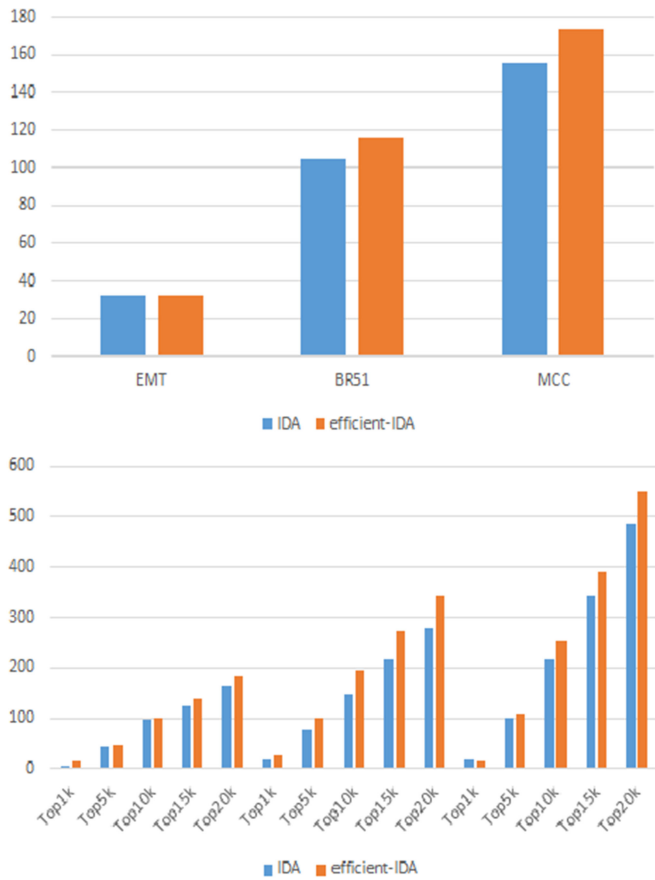


Fig. 5. Number of confirmed miRNA targets identified by IDA and efficient-IDA. The top figure: the top 100 targets of each of the miRNAs in a dataset are extracted for validation. The bottom figure: the top 1, 5, 10, 15, and 20 k interactions in each dataset (from left to right, NCI-60 (EMT), BR51, and MCC) are extracted respectively for validation.

Authorized licensed use limited to: Univ Politecnica de Madrid. Downloaded on October 13,2026 at 21:42:58 UTC from IEEE Xplore. Restrictions apply.

efficient-IDA helps improve the accuracy of the causal inference method (IDA).

7 Conclusions and Future Work

It is very important to explore causal relationships in real world high dimensional datasets. For instance, the discovered causal relationships can help elucidate the causes of fatal diseases from gene expression data. However, existing causal discovery method do not provide practical solutions to the problem. In this paper, we have developed an efficient algorithm for exploring causal relationships in high dimensional datasets based on the stable-PC algorithm and parallel computing framework. We modify the stable-PC algorithm and group the CI tests in the algorithm to enable the tests to be performed in parallel. Our method produces the causal relationships that are consistent with the stable-PC algorithm, but much more efficiently regarding the runtime. The experiment results from a wide range of commonly used real world datasets suggest the efficiency of the proposed method. parallel-PC also shows its usefulness by improving the efficiency and accuracy of IDA in inferring miRNA-mRNA regulatory relationships.

The effectiveness of the parallel-PC algorithm could be further improved with a job weighting and scheduling scheme. In the current version of parallel-PC, we divide the number of CI tests equally and distribute to different cores of the CPU. However, there is no guarantee that the execution time of different cores is the same. A smart job weighting and scheduling scheme would help speed up the algorithm.

Finally, a straightforward but useful future work is to apply the parallel-PC algorithm to other PC related causal discovery and causal inference methods. Most of the constraint based methods are based on the idea of the PC algorithm directly or indirectly. The FCI [2] algorithm and RFCI [68] are based on the PC algorithm to learn causal graphs which allows latent variables. The CCD algorithm [69] for learning Markov equivalent class utilises the PC algorithm as the first step. The parallelised versions of these algorithms would help bridging the gap between computer science theory and their applications in practice.

Acknowledgments

This work has been supported by the Australian Research Council Discovery Project DP140103617.

References

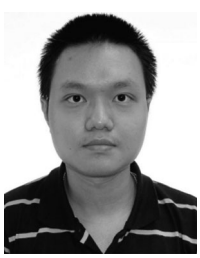
[1] J. Pearl, *Causality: Models, Reasoning, and Inference*. Cambridge, U.K.: Cambridge Univ. Press, 2000.
[2] P. Spirtes, C. Glymour, and R. Scheines, *Causation, Prediction, and Search*, 2nd ed. Cambridge, MA, USA: MIT Press, 2000.
[3] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. San Mateo, CA, USA: Morgan Kaufmann, 1988.
[4] R. Neapolitan, *Learning Bayesian Networks*. Upper Saddle River, NJ, USA: Pearson Prenctice Hall, 2004.
[5] C. W. Granger, "Investigating causal relations by econometric models and cross-spectral methods," *Econometrica: J. Econometric Soc.*, vol. ES-37, no. 3, pp. 424–438, 1969.
[6] C. A. Sims, "Money, income, and causality," *Amer. Econ. Rev.*, vol. 62, no. 4, pp. 540–552, 1972.
[7] R. W. Robinson, "Counting labeled acyclic digraphs," in *New Directions in the Theory of Graphs: Proc. 3rd Annu. Arbor Conf. Graph Theory (1971)*, New York, USA: Academic Press, 1973, pp. 239–273.
[8] T. Verma and J. Pearl, "Equivalence and synthesis of causal models," in *Proc. 6th Conf. Uncertainty Artificial Intell.*, 1990, pp. 220–227.
[9] M. Kalisch, M. Mächler, D. Colombo, M. H. Maathuis, and P. Bühlmann, "Causal inference using graphical models with the R package pcalg," *J. Statistical Softw.*, vol. 47, no. 11, pp. 1–26, 2012.
[10] M. Scutari, "Learning Bayesian networks with the bnlearn R package," *J. Statistical Softw.*, vol. 35, no. 3, pp. 1–22, 2010.
[11] X. Zhang, "Inferring gene regulatory networks from gene expression data by path consistency algorithm based on conditional mutual information," *Bioinf.*, vol. 28, no. 1, pp. 98–104, 2012.
[12] H. M. Maathuis, D. Colombo, M. Kalisch, and P. Buhlmann, "Predicting causal effects in large-scale systems from observational data," *Nature Methods*, vol. 7, pp. 247–249, 2010.
[13] T. D. Le, et al., "Inferring microRNA–mRNA causal regulatory relationships from expression data," *Bioinf.*, vol. 29, no. 6, pp. 765–771, 2013.
[14] J. Zhang, et al., "Inferring condition-specific miRNA activity from matched miRNA and mRNA expression data," *Bioinf.*, vol. 30, pp. 3070–3077, 2014.
[15] J. Zhang, et al., "Identifying direct miRNA-mRNA causal regulatory relationships in heterogeneous data," *J. Biomed. Informat.*, vol. 52, pp. 438–47, 2014.
[16] C. Silverstein, S. Brin, R. Motwani, and J. Ullman, "Scalable techniques for mining causal structures," *Data Mining Knowl. Discovery*, vol. 4, no. 2, pp. 163–192, 2000.
[17] I. Tsamardinos, C. F. Aliferis, A. Statnikov, and E. Statnikov, "Algorithms for large scale Markov blanket discovery," in *Proc. 16th Int. Florida Artificial Intell. Res. Soc. Conf.*, 2003, pp. 376–381.
[18] J. Li, T. D. Le, L. Liu, J. Liu, Z. Jin, and B. Sun, "Mining causal association rules," in *Proc. ICDM Workshop Causal Discovery*, 2013, pp. 114–123.
[19] J. Li, et al., "From observational studies to causal rule mining," *ACM Trans. Intell. Syst. Technol.*, vol. 7, no. 3, 2015, Art. no. 14.
[20] D. Colombo and M. H. Maathuis, "A modification of the PC algorithm yielding order-independent skeletons," *J. Mach. Learning Res.*, vol. 15, no.1, pp. 3741–3782, 2014.
[21] D. Dash and M. J. Druzdzel, "A hybrid anytime algorithm for the construction of causal models from sparse data," in *Proc. 15th Conf. Uncertainty Artificial Intell.*, 1999, pp. 142–149.
[22] A. Cano, M. Gómez-Olmedo, and S. Moral, "A score based ranking of the edges for the PC algorithm," in *Proc. 4th Eur. Workshop Probabilistic Graph. Models*, 2008, pp. 41–48.
[23] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," *Commun. ACM*, vol. 51, no. 1, pp. 107–113, 2008.
[24] D. Heckerman, D. Geiger, and D. M. Chickering, "Learning Bayesian networks: The combination of knowledge and statistical data," *Mach. Learn.*, vol. 20, no. 3, pp. 197–243, 1995.
[25] D. Edwards, *Introduction to Graphical Modelling*. New York, NY, USA: Springer, 2000.
[26] G. F. Cooper and E. Herskovits, "A Bayesian method for the induction of probabilistic networks from data," *Mach. Learn.*, vol. 9, no. 4, pp. 309–347, 1992.
[27] C. Glymour and G. Cooper, *Causation, Computation and Discovery*. Cambridge, MA, USA: MIT Press, 1999.
[28] D. M. Chickering, D. Geiger, and D. Heckerman, "Learning Bayesian networks is NP-hard," *Microsoft Res.*, Tech. Rep. MSR-TR-94-17, vol. 196, 1994.
[29] G. F. Cooper, "A simple constraint-based algorithm for efficiently mining observational databases for causal relationships," *Data Mining Knowl. Discovery*, vol. 1, no. 2, pp. 203–224, 1997.
[30] M. Kalisch and P. Buhlmann, "Estimating high-dimensional directed acyclic graphs with the PC-algorithm," *J. Mach. Learn. Res.*, vol. 8, pp. 613–636, 2007.
[31] H. Steck and V. Tresp, "Bayesian belief networks for data mining," in *Proc. 2nd Workshop Data Mining Data Warehousing als Grundl. Moderner Entscheidungsunterstützender Syst.*, 1999, pp. 145–154.
[32] J. Abelln, M. Gmez-olmedo, and S. Moral, "Some variations on the PC algorithm," in *Proc. 1st Eur. Workshop Probabilistic Graph. Models*, 2006, pp. 1–8.
[33] D. Margaritis and S. Thrun, "Bayesian network induction via local neighborhoods," in *Advances in Neural Information Processing Systems*, S. Solla, T. Leen, and K. R. Müller, Eds. Cambridge, MA, USA: MIT Press, 1999.

Authorized licensed use limited to: Univ Politecnica de Madrid. Downloaded on October 13,2026 at 21:42:58 UTC from IEEE Xplore. Restrictions apply.

[34] S. Yaramakala and D. Margaritis, "Speculative Markov blanket discovery for optimal feature selection," in *Proc. 5th IEEE Int. Conf. Data Mining*, 2005, pp. 1550–4786.
[35] C. F. Aliferis, A. Statnikov, I. Tsamardinos, S. Mani, and X. D. Koutsoukos, "Local causal and Markov blanket induction for causal discovery and feature selection for classification part I: Algorithms and empirical evaluation," *J. Mach. Learn. Res.*, vol. 11, pp. 171–234, 2010.
[36] I. Tsamardinos, C. F. Aliferis, and A. Statnikov, "Time and sample efficient discovery of Markov blankets and direct causal relations," in *Proc. 9th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2003, pp. 673–678.
[37] J. M. Peña, R. Nilsson, J. Björkegren, and J. Tegnér, "Towards scalable and data efficient learning of Markov boundaries," *Int. J. Approximate Reason*, vol. 45, no. 2, pp. 211–232, 2007.
[38] S. Fu and M. C. Desmarais, "Fast Markov blanket discovery algorithm via local learning within single pass," in *Advances in Artificial Intelligence*. New York, NY, USA: Springer, 96–107, 2008.
[39] Z. Jin, J. Li, L. Liu, T. D. Le, B. Sun, and R. Wang, "Discovery of causal rules using partial association," in *Proc. IEEE 12th Int. Conf. Data Mining*, 2012, pp. 309–318.
[40] J. Li, L. Liu, and T. D. Le, *Practical Approaches to Causal Relationship Exploration*. New York, NY, USA: Springer, 2015.
[41] M. Scutari, "Bayesian network constraint-based structure learning algorithms: Parallel and optimised implementations in the bnlearn R package," arXiv 1406:7648, 2014.
[42] Y. Chen, J. Tian, O. Nikolova, and S. Aluru, "A parallel algorithm for exact Bayesian structure discovery in Bayesian networks," 2014. [Online]. Available: http://arxiv.org/abs/1408.1664v1
[43] J. Ramsey, J. Zhang, and P. L. Spirtes, "Adjacency-faithfulness and conservative causal inference," arXiv:1206.6843, 2012.
[44] R Core Team, *R: A Language and Environment for Statistical Computing*. Vienna, Austria: R Foundation Statistical Comput., 2014.
[45] T. D. Le, L. Liu, J. Zhang, B. Liu, and J. Li, "From miRNA regulation to miRNA-TF co-regulation: Computational approaches and challenges," *Briefings Bioinf.*, vol. 16, pp. 475–96, 2015.
[46] D. Marbach et al.,"Wisdom of crowds for robust gene network inference," *Nature Methods*, vol. 9, pp. 796–804, 2012.
[47] D. P. Bartel, "MicroRNAs: Genomics, biogenesis, mechanism, and function," *Cell*, vol. 116, pp. 281–297, 2004.
[48] V. N. Kim and J. W. Nam, "Genomics of microRNA," *Trends Genetics*, vol. 22, pp. 165–173, 2006.
[49] A. J. Enright, B. John, U. Gaul, T. Tuschl, C. Sander, and D. S. Marks, "microRNA targets in Drosophila," *Genome Biol.*, vol. 5, pp. R1–R1, 2004.
[50] B. P. Lewis, C. B. Burge, and D. P. Bartel, "Conserved seed pairing, often flanked by adenosines, indicates that thousands of human genes are microRNA targets," *Cell*, vol. 120, pp. 15–20, 2005.
[51] A. Krek, et al., "Combinatorial microRNA target predictions," *Nature Genetics*, vol. 37, pp. 495–500, 2005.
[52] N. Rajewsky, "microRNA target predictions in animals," *Nature Genetics*, vol. 38, pp. S8–S13, 2006.
[53] H. Liu, et al., "Identifying mRNA targets of microRNA dysregulated in cancer: With application to clear cell Renal Cell Carcinoma," *BMC Syst. Biol.*, vol. 4, no. 1, 2010.
[54] I. Van der Auwera, R. Limame, P. Van Dam, P. Vermeulen, L. Dirix, and S. J. Van Laere, "Integrated miRNA and mRNA expression profiling of the inflammatory breast cancer subtype," *British J. Cancer*, vol. 103, pp. 532–541, 2010.
[55] Y. Lu, Y. Zhou, W. Qu, M. Deng, and C. Zhang, "A Lasso regression model for the construction of microRNA-target regulatory networks," *Bioinf.*, vol. 27, pp. 2406–2413, 2011.
[56] A. Muniategui, et al., "Quantification of miRNA-mRNA interactions," *PloS One*, vol. 7, 2012, Art. no. e30766.
[57] J. G. Joung, K. B. Hwang, J. W. Nam, S. J. Kim, and B. T. Zhang, "Discovery of microRNA-mRNA modules via population-based probabilistic learning," *Bioinf.*, vol. 23, pp. 1141–1147, 2007.
[58] D. H. Tran, K. Satou, and T. B. Ho, "Finding microRNA regulatory modules in human genome using rule induction," *BMC Bioinf.*, vol. 9, no. Suppl 12, 2008, Art. no. S5.
[59] J. C. Huang, et al., "Using expression profiling data to identify human microRNA targets," *Nature Methods*, vol. 4, pp. 1045–1050, 2007.
[60] B. Liu, J. Li, A. Tsykin, L. Liu, A. B. Gaur, and G. J. Goodall, "Exploring complex miRNA-mRNA regulatory networks by splitting-averaging strategy," *BMC Bioinf.*, vol. 19, pp. 1–19, 2009.
[61] T. D. Le, et al., "Inferring microRNA and transcription factor regulatory networks in heterogeneous data," *BMC Bioinf.*, vol. 14, no. 1, 2013.
[62] T. D. Le, J. Zhang, L. Liu, and J. Li, "Ensemble methods for MiRNA target prediction from expression data," *PloS One*, vol. 10, no. 6, 2015, Art. no. e0131627.
[63] H. M. Maathuis, M. Kalisch, and P. Buhlmann, "Estimating high-dimensional intervention effects from observational data," *Ann. Statistics*, vol. 37, no. 6, pp. 3133–3164, 2009.
[64] T. Vergoulis, et al., "TarBase 6.0: Capturing the exponential growth of miRNA targets with experimental support," *Nucleic Acids Res.*, vol. 40, pp. D222–D229, 2012.
[65] F. Xiao, Z. Zuo, G. Cai, S. Kang, X. Gao, and T. Li, "miRecords: An integrated resource for microRNA-target interactions," *Nucleic Acids Res.*, vol. 37, pp. D105–D110, 2009.
[66] H. Dweep, C. Sticht, P. Pandey, and N. Gretz, "miRWalk–database: Prediction of possible miRNA binding sites by walking the genes of three genomes," *J. Biomed. Inf.*, vol. 44, pp. 839–847, 2011.
[67] S. D. Hsu, et al., "miRTarBase update 2014: An information resource for experimentally validated miRNA-target interactions," *Nucleic Acids Res.*, vol. 42, pp. D78–D85, 2014.
[68] D. Colombo, M. H. Maathuis, M. Kalisch, and T. S. Richardson, "Learning high-dimensional directed acyclic graphs with latent and selection variables," *Ann. Stat.*, vol. 40, no. 1, pp. 294–321, 2012.
[69] T. Richardson, "A discovery algorithm for directed cyclic graphs," in *Proc. 12th Int. Conf. Uncertainty Artificial Intell.*, 1996, pp. 454–461.



Thuc Duy Le received the BSc and MSc degrees in pure mathematics from the University of Pedagogy, Ho Chi Minh City, Vietnam, in 2002 and 2006, respectively, and the BSc degree in computer science from UniSA, in 2010. He received the PhD degree in computer science (Bioinformatics) from UniSA, in 2014. He is a research fellow at the University of South Australia (UniSA). His research interests include bioinformatics, data mining, and machine learning.



Tao Hoang received the BSc degree in computer science from the National University of Singapore, Singapore, in 2012. He is currently working toward the PhD degree at the University of South Australia (UniSA). His research interests include health informatics, text mining, and machine learning.



Jiuyong Li received the PhD degree in computer science from Griffith University, Australia. He is a professor with the School of Information Technology and Mathematical Sciences, University of South Australia. His research interests include the field of data mining, privacy preserving, and bioinformatics. He has published more than 100 papers and his research has been supported by five Australian Research Council Discovery grants.

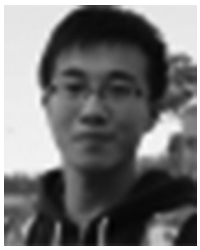


Lin Liu received the bachelor's and master's degrees in electronic engineering from Xidian University, China, in 1991 and 1994, respectively, and the PhD degree in computer systems engineering from UniSA, in 2006. She is a senior lecturer with the School of Information Technology and Mathematical Sciences, University of South Australia (UniSA). Her research interests include data mining and bioinformatics, as well as Petri nets and their applications to protocol verification, and network security analysis.

Authorized licensed use limited to: Univ Politecnica de Madrid. Downloaded on October 13,2026 at 21:42:58 UTC from IEEE Xplore. Restrictions apply.



Huawen Liu received the MSc and PhD degrees in computer science from Jilin University, China, in 2007 and 2010, respectively. He is an associate professor with the School of Mathematics, Physics and Information Engineering, Zhejiang Normal University, China. Since 2010, he has been with the Department of Computer Science, Zhejiang Normal University, China. His research interests include data mining, feature selection, and sparse learning for machine learning



Shu Hu received the BSc degree in electronics and information engineering from the North China Institute of Science and Technology, in 2013. He is a visiting researcher at the University of South Australia (UniSA). His research interests include data mining and machine learning.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/publications/dlib.

Authorized licensed use limited to: Univ Politecnica de Madrid. Downloaded on October 13,2026 at 21:42:58 UTC from IEEE Xplore. Restrictions apply.